

Proposal: Agent Roles — a portable role specification for multi-agent systems

I've spent the last several months working on a multi-agent open-source project, [claude_codex_bridge](#), and one recurring problem pushed me toward a separate spec project: [Agent Roles](#).

The problem: as agent setups grow, skills are easy to accumulate but hard to manage as a system. Each new skill mixes into a shared pool, tool dependencies blur, and every time you spin up a subagent for a specific function you end up re-explaining the same context through long ad-hoc prompts. There is no clean package boundary around “a specialist agent.”

The idea

A Role is a higher-level unit than a skill. Where a skill describes one reusable capability, a Role describes a complete specialist agent:

role memory + skills + tool dependencies + permission boundaries + host adapter metadata

The intent is that a Role is a static, portable definition: installable from a catalog, mountable into an agent host, and removable without polluting the main environment. Role source should not store project state or session history. Hosts may generate projection output from role source; that output should not get written back into the role.

Role is not a new concept

Custom GPTs in the ChatGPT Store and character cards in AI role-playing communities are both pointing at a similar need: stable, reusable agent identities with specific behaviors, knowledge, and interaction patterns. Product-specific persona systems — Claude Projects, Copilot instructions files, and similar — are also working toward this from inside specific platforms.

What those systems have not needed to address is the engineering packaging side: portability across different hosts, declared tool dependencies, lifecycle management (install / update / mount / unmount), and interoperability outside a single product's ecosystem. That is the gap I am trying to address with Agent Roles.

What exists now

The repo has a working preview:

- `specs/role-v1.md` and `specs/metadata-v1.md` describe the role directory shape and `role.toml` definition format
- an npm CLI (`agent-roles`, currently `0.1.1`) for installing and listing roles from a remote catalog
- a reference role (`agentroles.archi`, an architecture reviewer) as a concrete example of the format

The current spec is intentionally conservative for v0.1: permissions are advisory declarations, not enforced; cross-host binding is a concept rather than a required file format; runtime mounting and hot reload are deferred. My goal for this phase is to get the static Role definition shape right before adding lifecycle machinery on top.

Questions where I'd value outside input

On the definition format: `role.toml` as the role manifest feels right for human readability and minimal tooling needs, but I am not committed to it. Would a different format work better for the tooling you actually use?

On permissions: `[permissions]` in `role.toml` is currently a high-level advisory declaration (`read_files = true, write_files = false, network = false`). This makes the preview usable without requiring a full permission runtime, but it also means the declarations don't actually enforce anything. Is that the right trade-off for a preview spec, or does it make the whole permissions section misleading?

On scope: The spec intentionally describes the portable Role artifact rather than a marketplace or host runtime. I've tried to keep the core format host-neutral so that different agent hosts can implement adapters without being locked into one implementation. Does that framing hold up, or is it too abstract without a concrete host to anchor against first?

On the concept itself: I may be solving a problem that existing plugin systems already handle, or solving it at the wrong layer. If you see an obvious gap in the framing, I'd rather hear it early.

Project: <https://github.com/SeemSeam/agent-roles-spec>

If you work on agent platforms, coding-agent hosts, or multi-agent systems and have thoughts — including skeptical ones — I'd genuinely like to hear them.